Bandit Writeup

BY

Prasad Nikam

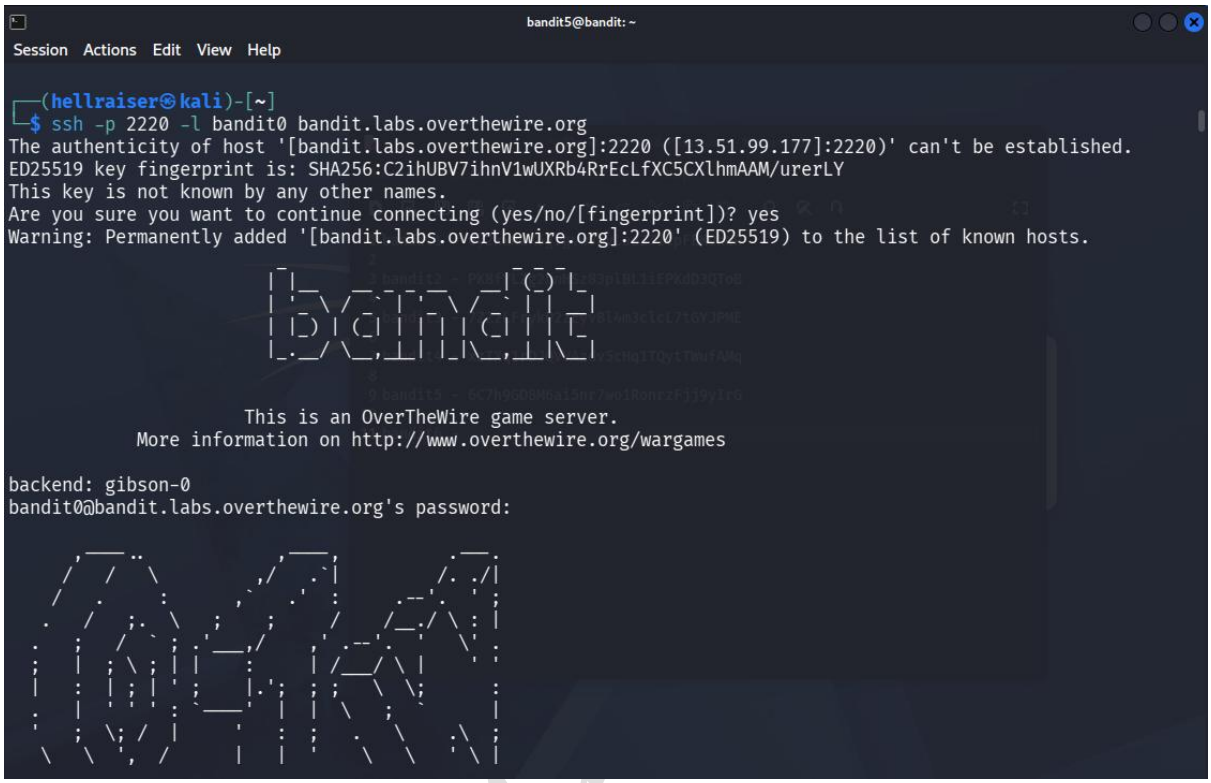
Prasad Nikam

Sr. No.	Title	Page No.
1	Level 0 - Bandit0	3
2	Level 1 - Bandit1	4
3	Level 2 - Bandit2	5
4	Level 3 - Bandit3	6
5	Level 4 - Bandit4	7
6	Level 5 - Bandit5	8
7	Level 6 - Bandit6	9
8	Level 7 - Bandit7	10
9	Level 8 - Bandit8	11
10	Level 9 - Bandit9	12
11	Level 10 - Bandit10	13
12	Level 11 - Bandit11	14
13	Level 12 - Bandit12	15
14	Level 13 - Bandit13	16
15	Level 14 - Bandit14	17
16	Level 15 - Bandit15	18
17	Level 16 - Bandit16	19
18	Level 17 - Bandit17	22
19	Level 18 - Bandit18	23
20	Level 19 - Bandit19	24
21	Level 20 - Bandit20	25
22	Level 21 - Bandit21	26
23	Level 22 - Bandit22	27
24	Level 23 - Bandit23	28
25	Level 24 - Bandit24	30
26	Level 25 - Bandit25	31
27	Level 26 - Bandit26	33
28	Level 27 - Bandit27	34
29	Level 28 - Bandit28	35
30	Level 29 - Bandit29	37
31	Level 30 - Bandit30	38
32	Level 31 - Bandit31	39
33	Level 32 - Bandit32	40
34	Level 33 - Bandit33	41

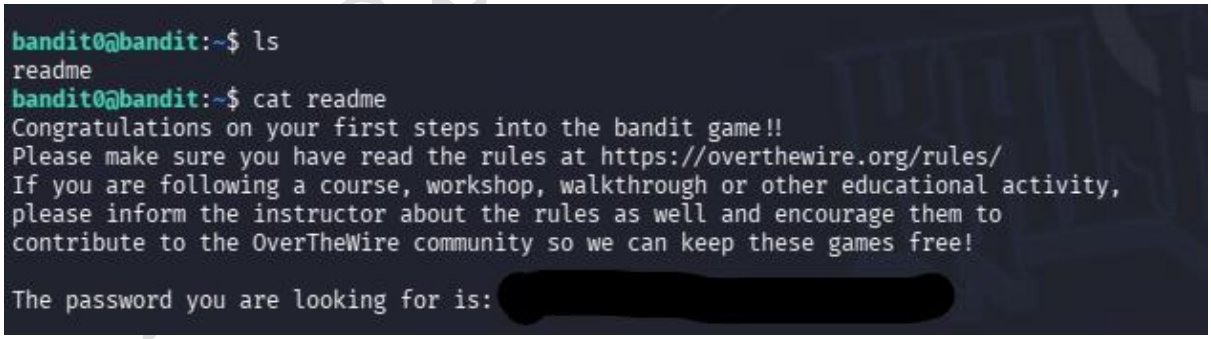
Prasad Nikam

Level 0 – Bandit0

According to the instruction on the page of Bandit the level 0 can be accessed through the given password and login id i.e. bandit0. So, we need to log on to using the given port number, login id, password, and URL using command "`ssh -p 2220 -l bandit0 bandit.labs.overthewire.org`" this command will used further for every advancement we do in the levels.



After accessing the we can perform various actions like listing directories and files and can open them as well to check whether they contain the password or not.



As we can see this system contains a file and upon opening it contains a message congratulating us for reaching the destination with the password for the next level which is Bandit1.

Level 1 – Bandit1

As we did in the previous level we need to log onto the next system which is Bandit1, using the same command but the login id will change to “bandit1” this will apply for all the next levels we will face.



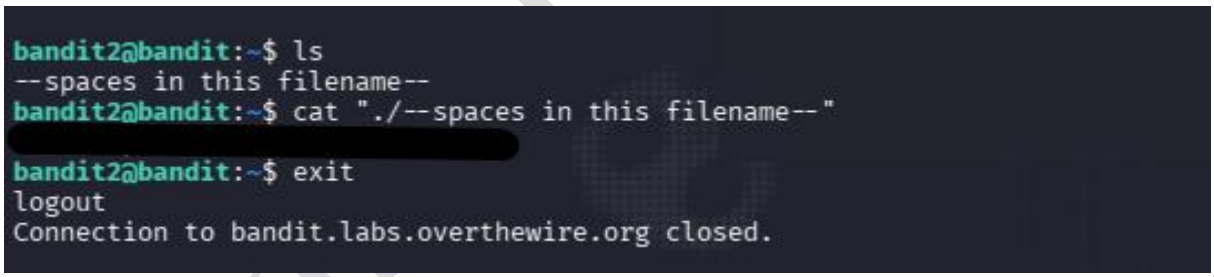
Now that we have accessed the system, we will do the same steps first that is listing using “ls” command. As we can see that there is a file named “-” in the system which contains the password we are looking for but if just print it according to regular command i.e. is “cat –” then it show us error so we need to add “./” in front of the filename so it will show us the required password for logging in Bandit2.



Level 2 – Bandit2



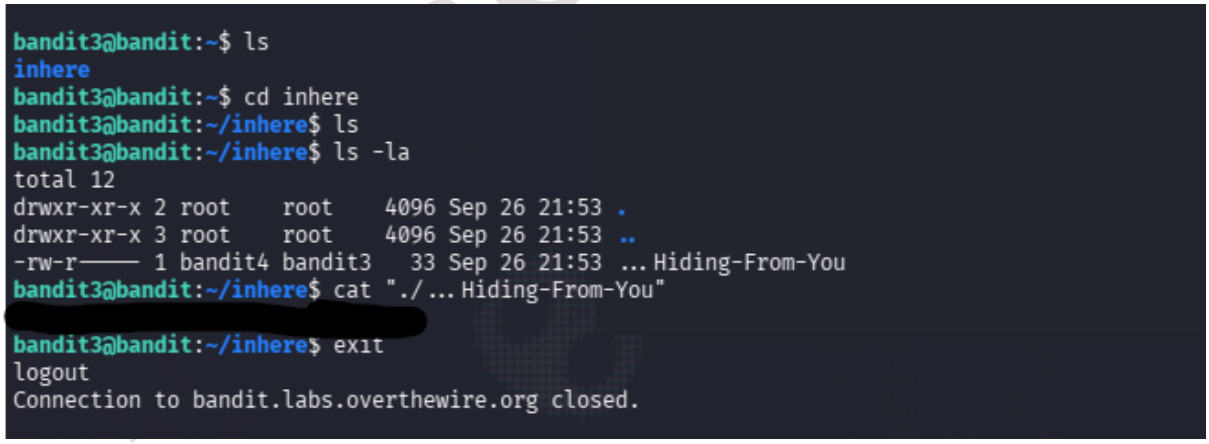
As we log in the system and check for the files, we have a file named "—spaces in this filename--". Which can be seen using the previous format but as it has spaces in between so we need to provide the filename in double quotes, so that we can have the password it has for the next level i.e. Bandit3.



Level 3 – Bandit3



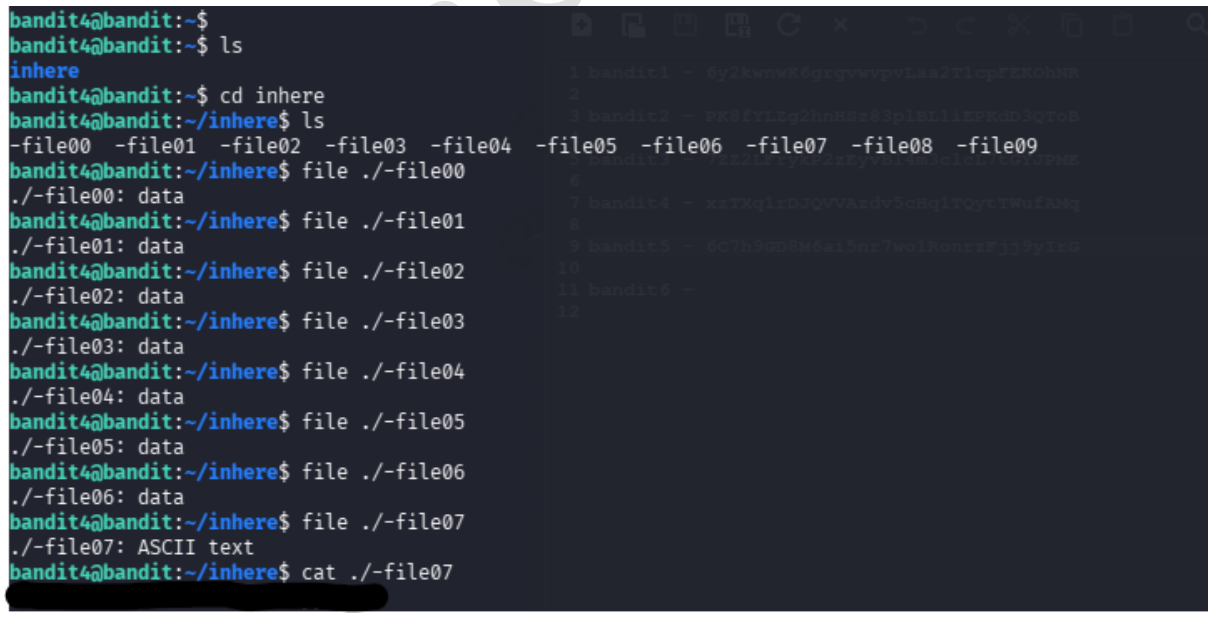
The instructions for this level are that there is a hidden file in the directory name “inhere”. So, we need to check the directory present first. So, we use “ls” and “cd” to list and then traverse to that directory. Now if check using the “ls” command it does not list any file or directory for that we need to add options for this i.e. “-la” l for detailed list and a for whole list. Then we can use the “cat” command for the password of Bandit4.



Level 4 – Bandit4

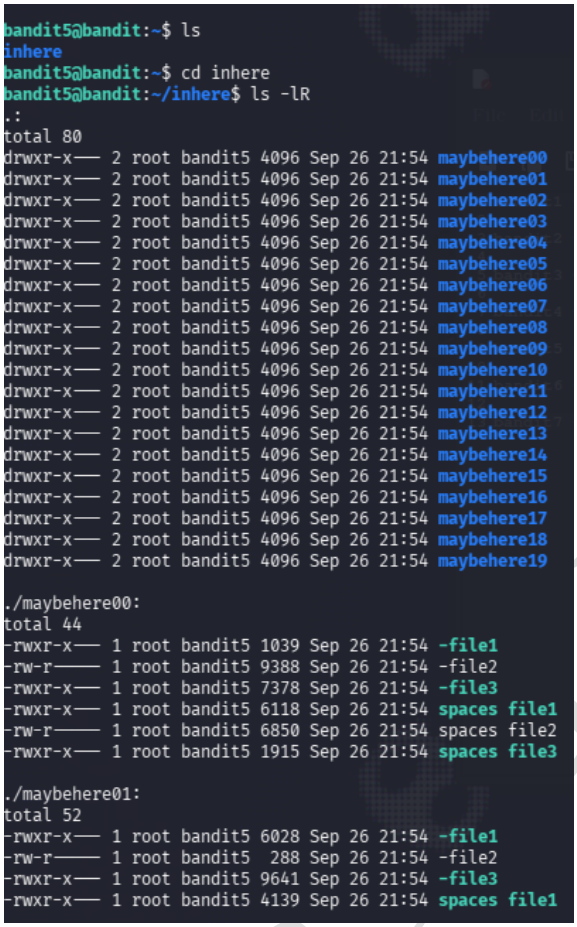


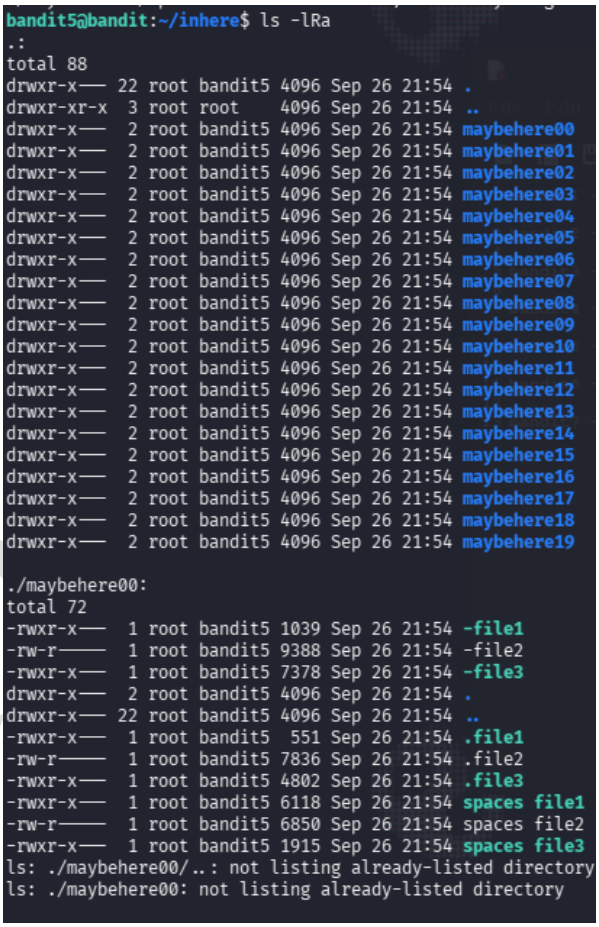
The instruction for this level state that the password for bandit5 is in the only human readable file in the directory “inhere”. So, we traverse to “inhere” and the directory contains 10 files. We can determine which file contains what kind of information in it. We need a file which contains ASCII text in it. That is “-file07”, which upon viewing provides us with the password.



Level 5 – Bandit5

According to the description of the file mentioned on the site, the file is human-readable, non-executable, and 1033 bytes in size, also located somewhere in "inhere" directory. So, we need to search it in the directories which may contain this file. We used the "`ls -lR`" to do so here "R" stands for recursive search, if just enter "ls -l" it will show the details of only the listed directories within the directory, but what it does now is show information regarding whatever is in these directories.





But upon further inspection, we can locate a file which matches the description as is so we need to "a" as well to get hidden files and after doing so there more files that were not present at first, and then we can find the file which we need and the password within it.



Level 6 – Bandit6

The instructions for this level are that the file is located somewhere on the server with properties: owned by user bandit7 and owned by group bandit6. So first we need to check if the system contains any directories or files, which showed no result.

```
bandit6@bandit:~$ ls
bandit6@bandit:~$ ls -la
total 20
drwxr-xr-x   2 root root 4096 Oct 25 13:36 .
drwxr-xr-x 150 root root 4096 Oct 25 13:40 ..
-rw-r--r--   1 root root  220 Feb 13  2026 .bash_logout
-rw-r--r--   1 root root 3851 Oct 25 13:21 .bashrc
-rw-r--r--   1 root root  807 Feb 13  2026 .profile
```

So, we can use the command "`find / -user bandit7 2>/dev/null`", this shows result for files or directory created by user bandit7 without those which we do not have access for i.e. "`2>/dev/null`". When we inspect the list of files and directories that are created by user bandit7, there is a file bandit7.password which has the password for the next level.

```
bandit6@bandit:~$ find / -user bandit7 2>/dev/null
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/cgroup.procs
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/cgroup.threads
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cgroup.events
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.events
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/io.pressure
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cgroup.procs
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.events.local
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.swap.peak
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.swap.current
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.swap.max
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.zswap.current
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cpu.weight
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.swap.events
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cgroup.max.descendants
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cgroup.stat.local
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cpu.stat
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cpu.weight.nice
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.pressure
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.current
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/pids.current
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.stat
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/pids.events
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.low
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cpu.pressure
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cgroup.type
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cgroup.stat
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/memory.swap.high
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/app.slice/cpu.idle
```

```
/sys/fs/cgroup/user.slice/user-11007.slice/user@11007.service/memory.reclaim
/run/user/11007
/var/lib/dpkg/info/bandit7.password
/etc/bandit_pass/bandit7
```

```
bandit6@bandit:/$ cat /var/lib/dpkg/info/bandit7.password
[redacted]
bandit6@bandit:/$
```

Level 7 – Bandit7

The instructions state that the password for the next level is stored in the file named “data.txt” near the word “millionth”. So, we check the listing and there is a file named “data.txt”, but upon viewing the file there are passwords in it.

```
bandit7@bandit:~$ ls
data.txt
bandit7@bandit:~$ cat data.txt
longhand        nW3wqwr9Gz9QmQ7FGJdJinb6hnBSADR1
emancipation    4ATIWSH3IKhMJenTl6tcdAnEQCqsDRJg
damson's        IWMOM71xD0osTLm4duVunnBDU6047UaF
deviants        hv2WJvHQWDLw4LmPgJlGHyuCj0Cds1hG
Magoo's u9wJ4xubncP4nZ81Por6RNydOzyzxh1f
macro   4nbuGRA16kuoTVwk3oX0HkC6Ok7HPzSt
finessing       kdt5gsPQWdzUjXei0GdsPVbfBAcRZ3m7
daisy   joSXNDPiTSYa5yC56XMYXMznqFyuShmR
shoddiness      gzi2VuMI8ygIYTJ7CU94DrQfB6qh1ctm
madman's        TkxtLm3i7GTqSyNextUiBMBdWBnPSD97
streetlight     O7KK0jrciPf3PpY8BMg1GxQiWTmk7GPq
cruelty nTxFy8EBrfgjmlMHzQlkUoZDMJnvgtQp
pathetic        2eUeyjWCnZaw6H3aukh9B3wVR9ENOBhG
ankles  rkiQl3vTT7ohwsuyfYJfcBIQu3YugRXu
luaus   S6q6orviY51KXJ7xSg3JM2pt4BTlOqWh
Haiti's H6ItBRqq4GWg6iXovCvv2zzsW3FRjul5
scum's  qwQObM2Xw05B7XUtvNsZoYIbD2Pw8n44
anesthetizes    kyBgmmfLwB4fjK1sBetyZbyrGIkKWWir
Taft's  BBAADUbaxzkeRbmPHJ0kHXVFjtMeoNBu
hares   UEIPGUjDnFSRQYXvnxKuVFVoH7TsuhdP
phenotype       cc0XHM2pll84jG7Ei8zDCAVF0owlTUHn
commuted        p1MGnOYzdCF8VIj2bdk3FXKhzVb4dROM
propagated      BwariVco7EC9X1MxzpmK0KFjguRTOZW0
upturn  7Te6hogcEyJ5RA9UzJoITMztF7PMICVj
Whistler        CetV2Kdi9XcaZNdN28dqCK2aUuf18qzR
teeth's O4LVpkD4VTEJj3bgykwbxbkgSrHKcpjN
Kuibyshev       aJV6Fqbsq1WcEc0b1pAuAStQKy4tcBaD
phones  OO4RfJcBhYgiWOPVIDbfGr43PbEjfMaL
```

So, we need to check for the particular word “millionth” by using the grep command i.e. “`grep -i “millionth” data.txt`” which then shows the line which contains the word millionth and password next to it.

```
bandit7@bandit:~$ grep -i "millionth" data.txt
millionth       [redacted]
bandit7@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Level 8 – Bandit8

The instructions state that the password for the next level is stored in the file "data.txt" and is the only line of text that occurs only once. So, we list the files and check it which contain multiple lines which may be recurring so we need to sort the data (ASCII text).

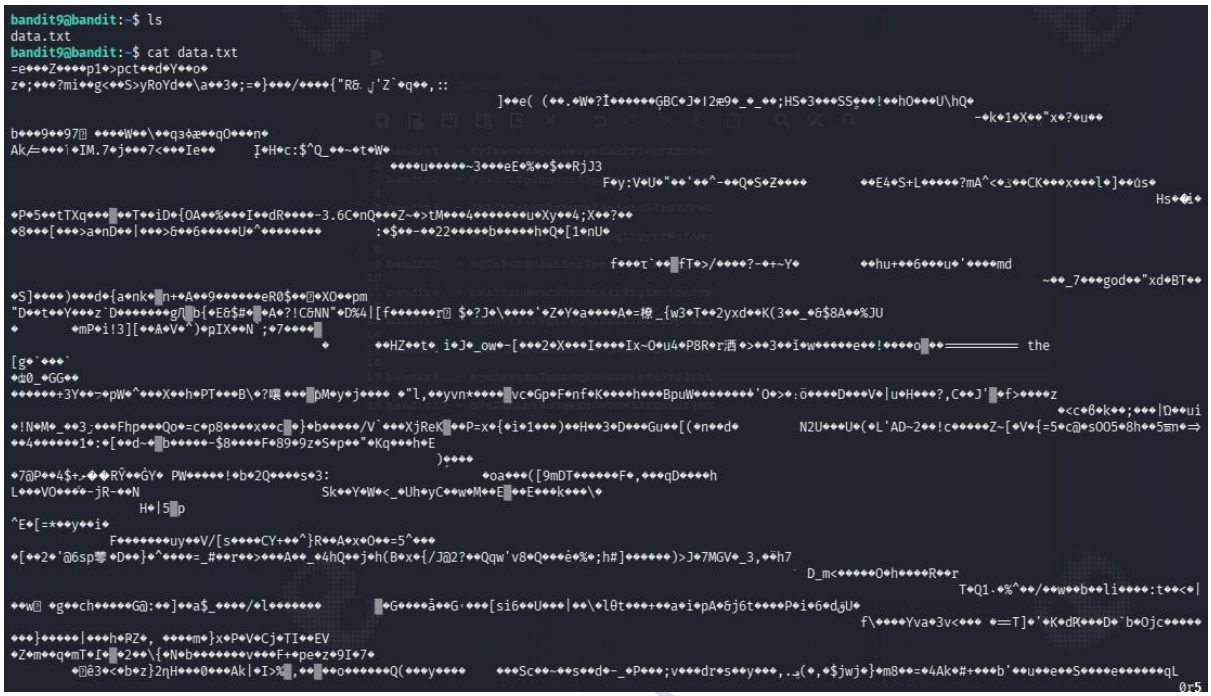
```
bandit8@bandit:~$ ls
data.txt
bandit8@bandit:~$ cat data.tx
cat: data.tx: No such file or directory
bandit8@bandit:~$ cat data.txt
D2kVzeYzDTiYvc5MAar8s6lhio9hVlhj
j05JStSDeBNI9RYDAkm9CKqfhcvoVj2W
D2kVzeYzDTiYvc5MAar8s6lhio9hVlhj
Brwn4ncMgea7KUpN62G6Vc9e9UWkeslj
j05JStSDeBNI9RYDAkm9CKqfhcvoVj2W
LQcvJtG0UM4wHUDRYmrrBcaKLTlHbYoc
ozYSsK0cbFPStgRGUkDl8nygTXuBFiF3
lQzpHjmKT8xD2fozwv9uLSnULT19o45s
BOLHFpwx8BtrjjXIAsoU8iAjyUqLocXJ
3rctgyGhkDw4T0uRflopilcpEh0WZUTz
PyR2sjbFg5Skgp5syuEy0S4Qd5aLlBg7
C1SWpsPguDyfbUr7tQQQDuppcN4rpNgI
brDx90PtNsNnA3wPprwCZdCEzstziX1u
Gite9W42d3QDStoYBFDuGEU7V9LnH4d7
jdREne9ep9XUxqlaghvKC6gvRxf7Wym9
KENvsezJTu8tkFdivY47qtktfG0GdbVT
HS3Y9cb6XoGM4ubLzxfSHzxl96cGXB12
brDx90PtNsNnA3wPprwCZdCEzstziX1u
HS3Y9cb6XoGM4ubLzxfSHzxl96cGXB12
N7hC6Qwy2jMBNTgdxWkBgDCGUSkmHEyl
iGHkjk7nlCK7lb7WNBqJFArYSBjQAYad
6YSOcAu3hypcX4iH5R3bSh0ksrwEE8mB
Q8wbWNC1dJSR6T9nV6xtps84F9TPrEQ5
VyEcL3UriZg8qP53MkCqiSHgtP2nhHC2
GldRqMJPqgqSC5VLf5xU8TIz9KuH9kyw
Nq6KpmWyCSJwvfF6wHQlFStzJOI1MnNv
30ppOzAmY6Nh8tebHjEfVG4uDr4NyVph
nab8Xvb9SBUgVdoJkrziqH6uD3EofWw6
vJJ5FDyR3fMzGIsFBsjLZxNN5gMkr63r
ncEe99YJ9NmbaWCCRBu8tN8dTNCeqhVZ
w1qam4MTdxmO5s74sy3DPsmRG0z7EYlB
4CZBSh4uEIY7BcPXnoXOev6ycb6sYHnY
R4gib1qZ6tpE0nM4CfEh0L57LKyZwqq3
XEzHEFkz4r2vUrIuwORVLaLrN5kO1yuj
TKSt70xHTjiNY5Fg7HPWmFJxnjEh3PTT
```

To do so we need to use the command "`sort data.txt | uniq -u`" which will list the lines which are unique i.e. non-recurring in the file. This listed one line which is our password for the next level i.e. Bandit9.

```
bandit8@bandit:~$
bandit8@bandit:~$ sort data.txt | uniq -u
[redacted]
bandit8@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Level 9 – Bandit9

The instruction for this level is that the password for the next level is stored in the file named “data.txt” in one of the few human-readable strings, preceded by several ‘=’ characters. When we check the file, it consists of data i.e. non-human readable.



Now to get what we want we can use the strings command i.e. “`strings data.txt | grep -E “^={2,}”`” this will show strings available in the file which are followed after 2 or more ‘=’. The result shows ‘=’ highlighted because of ‘-E’ and the strings which has the password.

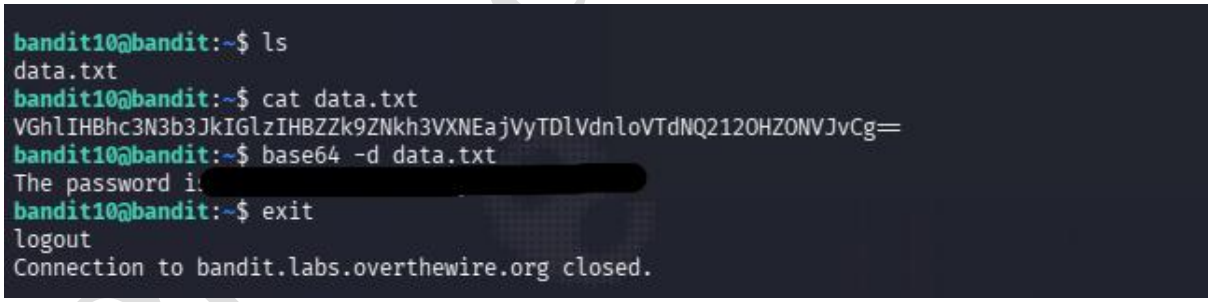


Level 10 – Bandit10

The instruction for this level is that the password is stored in file ‘data.txt’ which contains base64 encoded data.



To check for the file, we use “ls” and check for the data in it and it is whole ASCII text and we know that the password is not that long according to the previous levels. So, we can use the command “`base64 -d data.txt`” this will the decode the base64 encoded data in the file ‘data.txt’ i.e. “The password is _”

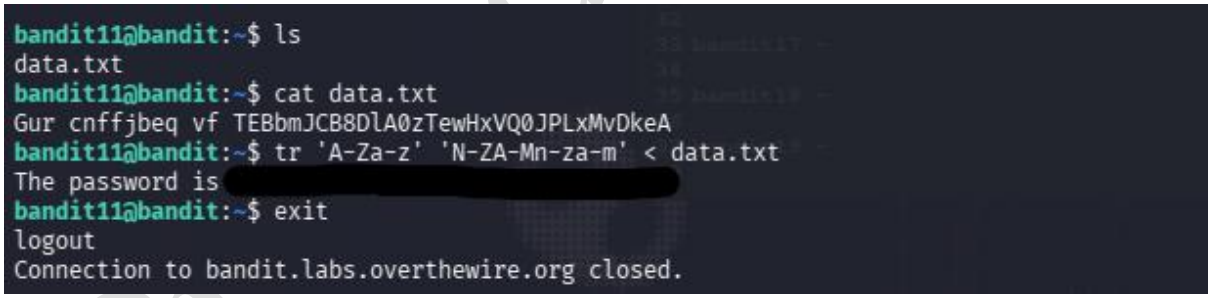


Level 11 – Bandit11

The instruction for this level is that the password is rotated by 13 positions for all letters lowercase (a-z) and uppercase (A-Z). So, the letters from a-m, and n-z are swapped.



Therefore, when we check the file "data.txt" everything is gibberish. So, we need to use the command "`tr 'A-Za-z' 'N-ZA-Mn-za-m' < data.txt`". This command swaps the designated characters that are to be shifted by 13 positions, which will show us the real text which we are supposed to see.



Level 12 – Bandit12

The instructions for this level are that the file 'data.txt' which consists of the password is actually a hexdump and it is been compressed several times as well. So, we need to decompress the file multiple times to get the actual password.

First, we start by making a temporary directory where we can perform this decompression process by using the command "`mydir=$(mktemp -d)`" following by the command "`cd $mydir`" which will move us to that directory for further commands and procedure. Then we will move the 'data.txt' file here to this directory and convert it to its original form i.e. binary and rename it to 'data.bin' by using the command "`xxd -r data.txt data.bin`". Then we check for the file type of 'data.bin' using command "`file data.bin`" which showed us its type to be gzip, we then unzip it by renaming it to '.gz' extension and extract it using "`gunzip data.gz`". then we repeat the same step for this file then converting it back to data.bin and check the file type again. It shows the file to be bzip2, so we repeat the process again by renaming it to and extract using the command "`bzip2 -d data.bz2`" then repeating the process by renaming it again to 'data.bin' then checking file type again.

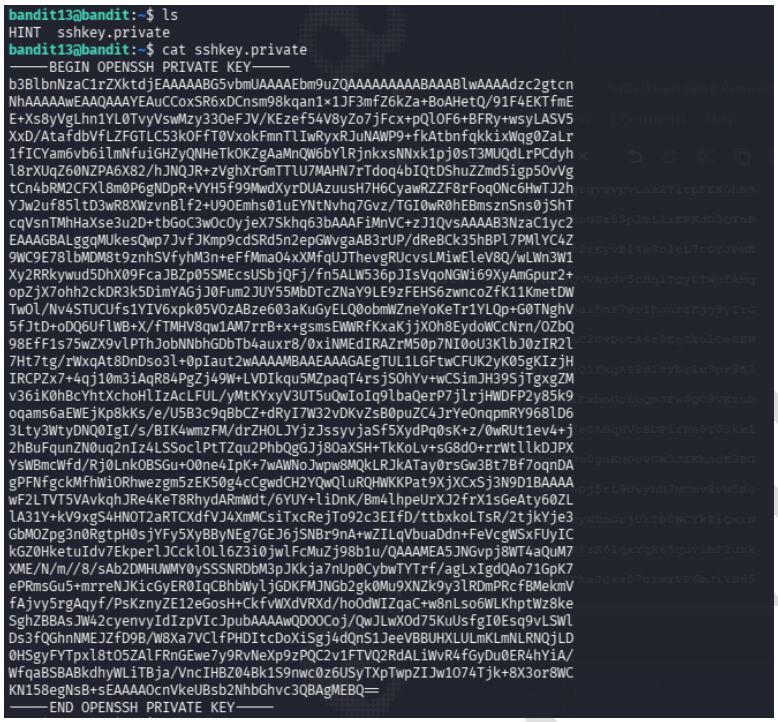
```
bandit12@bandit:~$ ls
data.txt
bandit12@bandit:~$ mydir=$(mktemp -d)
bandit12@bandit:~$ cd $mydir
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ cp ~/data.txt .
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ ls
data.txt
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ xxd -r data.txt data.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ file data.bin
data.bin: gzip compressed data, was "data2.bin", last modified: Sun Oct 25 13:37:03 2026, max compression, from Unix, original size modulo 2^32 585
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data.bin data.gz
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ gunzip data.gz
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data data.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ file data.bin
data.bin: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data.bin data.bz2
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ bzip2 -d data.bz2
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data data.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ file data.bin
data.bin: gzip compressed data, was "data4.bin", last modified: Sun Oct 25 13:37:03 2026, max compression, from Unix, original size modulo 2^32 20480
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data.bin data.gz
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ gunzip data.gz
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data data.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ file data.bin
data.bin: POSIX tar archive (GNU)
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data.bin data.tar
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ tar -xf data.tar
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ ls
data.tar  data.txt  data5.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data5.bin data.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ file data.bin
data.bin: POSIX tar archive (GNU)
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data.bin data.tar
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ tar -xf data.tar
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ ls
data.tar  data.txt  data6.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data6.bin data.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ file data.bin
data.bin: bzip2 compressed data, block size = 900k
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data.bin data.bz2
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ bzip2 -d data.bz2
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data data.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ file data.bin
```

This time the file type is POSIX tar archive, we then change its extension to '.tar' then extract it using the command "`tar -xf data.tar`" then check for what the file is named after extraction for the renaming it to 'data.bin' again to check for file type. We need to repeat this for every time the there is an archive file type till the file type is ASCII text. Then view the file for the password within it.

```
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ file data.bin
data.bin: POSIX tar archive (GNU)
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data.bin data.tar
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ tar -xf data.tar
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ ls
data.tar  data.txt  data8.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data8.bin data.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ file data.bin
data.bin: gzip compressed data, was "data9.bin", last modified: Sun Oct 25 13:37:03 2026, max compression, from Unix, original size modulo 2^32 49
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data.bin data.gz
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ gunzip data.gz
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ mv data data.bin
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ file data.bin
data.bin: ASCII text
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ cat data.bin
The password is [redacted]
bandit12@bandit:/tmp/tmp.DEprvZLBuQ$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Level 13 – Bandit13

The instruction for this level is that the password is stored in file "/etc/bandit_pass/bandit14" which can only be accessed by the user bandit14, but we can not access bandit14 as we don't have password for it, but we do have a way i.e. by using the ssh key for it using the command "`ssh -i sshkey. private bandit14@bandit.labs.overthewire.org -p 2220`" and can copy the key into a file at local and use it.

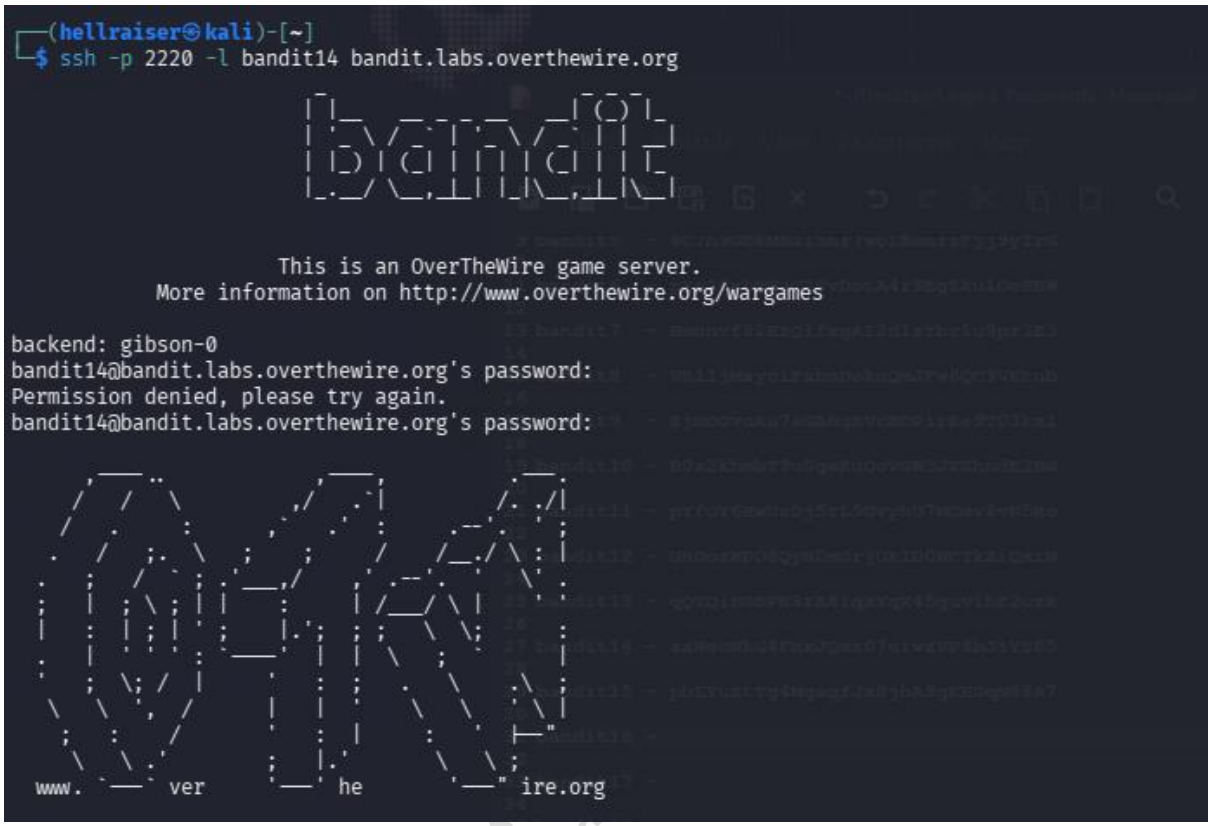




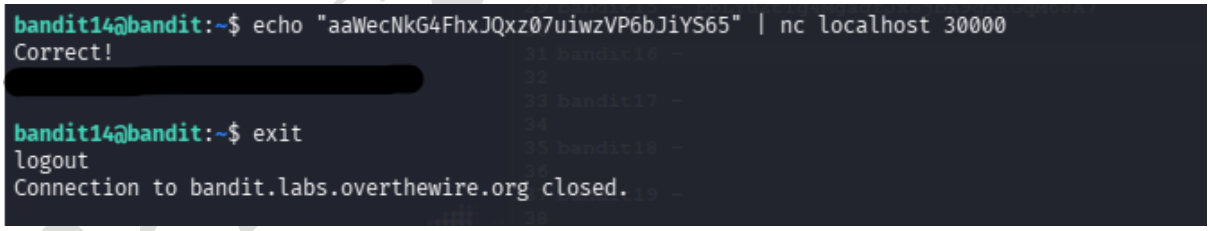

```
bandit14@bandit:~$ cd ../..
bandit14@bandit:/$ ls
behemoth  boot  drifter  formulaone  krypton  lib32  libx32      manpage  media  narnia  proc  run   snap  sys  usr     var
bin       dev   etc      home        lib      lib64  lost+found  maze     mnt    opt     root  sbin  srv   tmp  utumno  vortex
bandit14@bandit:/$ cd /etc/bandit_pass
bandit14@bandit:/etc/bandit_pass$ ls
bandit0   bandit11  bandit14  bandit17  bandit2   bandit22  bandit25  bandit28  bandit30  bandit33  bandit6  bandit9
bandit1   bandit12  bandit15  bandit18  bandit20  bandit23  bandit26  bandit29  bandit31  bandit4   bandit7
bandit10  bandit13  bandit16  bandit19  bandit21  bandit24  bandit27  bandit3   bandit32  bandit5   bandit8
bandit14@bandit:/etc/bandit_pass$ cat bandit14
[redacted]
bandit14@bandit:/etc/bandit_pass$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Level 14 – Bandit14

The instructions for this level are that the password for this level is the password for the next level i.e. Bandit15 but the port is changed to 30000. The way to retrieve the password for this level is by going 2 directories back and going into the "/etc/bandit_pass/bandit14".



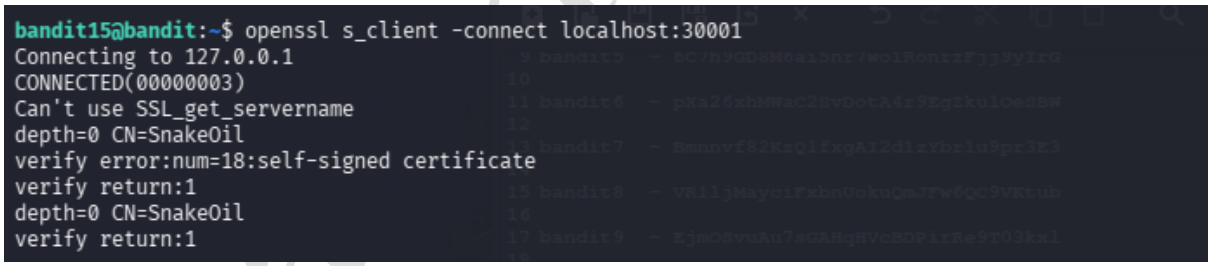
Then we use the "`echo "level password" | nc localhost 30000`" which will ping the port to get the password for the next level i.e. bandit15.



Level 15 – Bandit15

The instructions for this level are that the password for the next level can be retrieved by submitting the password for this level to localhost on port 30001 using ssl. Which can be done using command "`openssl s_client -connect localhost:30001`" which will ask for an input if we enter the password for this level, it will present us with password for next level i.e. Bandit16.







Level 16 – Bandit16

The instructions for this level are that the password for the next level can be obtained by submitting the password for this level to a port listening from 31000-32000 on localhost using ssl. So first we scan the listening ports using command "`nmap -p 31000-32000 localhost`" which shows the active listening port then we check the individual ports using command "`openssl s_client -connect localhost:31046`", here we just need change the ports accordingly.

```
bandit16@bandit:~$ nmap -p 31000-32000 localhost
Starting Nmap 7.98 ( https://nmap.org ) at 2026-10-28 09:19 +0000
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00014s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE
31046/tcp open  unknown
31518/tcp open  unknown
31691/tcp open  unknown
31790/tcp open  unknown
31960/tcp open  unknown

Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds
```

```
bandit16@bandit:~$ openssl s_client -connect localhost:31046
Connecting to 127.0.0.1
CONNECTED(00000003)
4067E7F7FF7F0000:error:0A0000F4:SSL routines:ossl_statem_client_read_transition:unexpected message:../ssl/statem/statem_clnt.c:423:
---
no peer certificate available
---
No client certificate CA names sent
Negotiated TLS1.3 group: <NULL>
---
SSL handshake has read 1533 bytes and written 1540 bytes
Verification: OK
---
New, (NONE), Cipher is (NONE)
Protocol: TLSv1.3
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
```

This port acts different from previous port but after providing the password it closes itself.

```
bandit16@bandit:~$ openssl s_client -connect localhost:31518
Connecting to 127.0.0.1
CONNECTED(00000003)
Can't use SSL_get_servername
depth=0 CN=SnakeOil
verify error:num=18:self-signed certificate
verify return:1
depth=0 CN=SnakeOil
verify return:1
---
Certificate chain
 0 s:CN=SnakeOil
   i:CN=SnakeOil
   a:PKEY: RSA, 4096 (bit); sigalg: sha256WithRSAEncryption
   v:NotBefore: Jun 10 03:59:50 2024 GMT; NotAfter: Jun  8 03:59:50 2034 GMT
---
```

```
---
read R BLOCK
[redacted]

KEYUPDATE
closed
```

```
bandit16@bandit:~$ openssl s_client -connect localhost:31691
Connecting to 127.0.0.1
CONNECTED(00000003)
4067E7F7FF7F0000:error:0A0000F4:SSL routines:ossl_statem_client_read_transition:unexpected message:
---
no peer certificate available
---
No client certificate CA names sent
Negotiated TLS1.3 group: <NULL>
---
SSL handshake has read 1533 bytes and written 1540 bytes
Verification: OK
---
New, (NONE), Cipher is (NONE)
Protocol: TLSv1.3
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
```

Where as this port acts differently and takes password as many time as you prompt the password. So, we check this port using option '-quiet' which gives a ssh key which can be used instead of password for the next level. We store it locally on our device and use it for logging in the next level. We can do so using command "`ssh -I sshkey.private bandit17@bandit.labs.overthewire.org -p 2220`".

```
bandit16@bandit:~$ openssl s_client -connect localhost:31790
Connecting to 127.0.0.1
CONNECTED(00000003)
Can't use SSL_get_servername
depth=0 CN=SnakeOil
verify error:num=18:self-signed certificate
verify return:1
depth=0 CN=SnakeOil
verify return:1
---
Certificate chain
 0 s:CN=SnakeOil
   i:CN=SnakeOil
   a:PKEY: RSA, 4096 (bit); sigalg: sha256WithRSAEncryption
   v:NotBefore: Jun 10 03:59:50 2024 GMT; NotAfter: Jun  8 03:59:50 2034 GMT
---
```

```
---
read R BLOCK
[redacted]
KEYUPDATE
[redacted]
KEYUPDATE

Wrong! Please enter the correct current password.
closed
```

```
┌──(hellraiser㉿kali)-[~]
└─$ ssh -i sshkey.private bandit17@bandit.labs.overthewire.org -p 2220
Warning: Identity file sshkey.private not accessible: No such file or directory.
                         _                     _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|


                      This is an OverTheWire game server.
            More information on http://www.overthewire.org/wargames
```

Then we retrieve the password from file '/etc/bandit_pass/bandit17' using the 'cat' command.

```
bandit17@bandit:~$ cat /etc/bandit_pass/bandit17
[redacted]
bandit17@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Prasad Nikam

Level 17 – Bandit17

The password for the next level is stored in passwords.new file which is the only line changed from file passwords.old. So, we can use the command "`diff passwords.old passwords.new`" which will tell which line is different in both the file.





Level 18 – Bandit18

The instruction for this level is that we cannot login into this level as someone has modified the '.bashrc' file, whereas the password is stored in file 'readme' on home directory.




```
Byebye !
Connection to bandit.labs.overthewire.org closed.
```

So, we can check the contents of the file using command "`ssh -p 2220 -l bandit18 bandit.labs.overthewire.org "cat *"`" which helps us view the file on home directory before the bashrc logs us out.



Level 19 – Bandit19

The instruction for the next level is that the password for the next level is the directory '/etc/bandit_pass/bandit20', but it can only be accessible by the user 'bandit20'. There is a file on the home directory named 'bandit20-do', which is a setuid file i.e. allows users to run an executable with the file system permission of the executable's owner or group respectively. Therefore, we can view the file '/etc/bandit_pass/bandit20' using this setuid, using command "`./bandit20-do cat /etc/bandit_pass/bandit20`".




```
andit19@bandit:~$ file bandit20-do
bandit20-do: setuid ELF 32-bit LSB executable, Intel i386, version 1 (SYSV),
99f568ae40f1caca2a9e937892, for GNU/Linux 3.2.0, not stripped
bandit19@bandit:~$ ./bandit20-do
Run a command as another user.
  Example: ./bandit20-do whoami
bandit19@bandit:~$ ./bandit20-do whoami
bandit20
bandit19@bandit:~$ ./bandit20-do cat /etc/bandit_pass/bandit20
[redacted]
bandit19@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Level 20 – Bandit20

The instruction for this level is that there is a setuid binary in the homedirectory that makes a connection to localhost on the port you specify as a commandline argument. It then reads a line of text from the connection and compares it to the password in the previous level (bandit20). If the password is correct, it will transmit the password for the next level (bandit21).

```
bandit20@bandit:~$ ls
suconnect
bandit20@bandit:~$ ./suconnect 1337
Read: 4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA
Password matches, sending next password
bandit20@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

For this we need two terminals and we need to log bandit20 on both. Then on first terminal we will run command "`nc -l 1337`" which will enable a listening port 1337 and simultaneously run command "`./suconnect 1337`" now if we prompt the current level password on the first terminal then it will the password the next level if the password matches.

```
bandit20@bandit:~$ nc -l 1337
[redacted]
bandit20@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Prasad Nikam

Level 21 – Bandit21



The instruction for this level is that a program is running automatically at regular intervals from cron. There is configuration in '/etc/cron.d/' and we are to check what that file is executing. So, we first need to change directory to '/etc/cron.d/' and check the files, there is a file that is named 'cronjob_bandit22' which is regarding to the next level, upon viewing the file it is executing a shell file from directory '/usr/bin/cronjob_bandit22.sh', this shell is the one which is executing the command "`chmod 644 /tmp/t7061lds9S0RqQh9aMcz6ShpAoZKF7fgv`" at regular intervals and the contents of the file '/etc/bandit_pass/bandit22' are stored in it. When we view this file, it shows us the password for the next level i.e. bandit22.



Level 22 – Bandit22

The instruction for this level is the same as last level bandit21. So, we need to same i.e. change directory to '/etc/cron.d' and view the 'cronjob_bandit23' which shows different contents than previous level. It has 2 commands which we will have to run to get to password. First is "`whoami`" which in our case would be bandit23 the next command is "`echo I am user bandit23 | md5sum | cut -d ' ' -f 1`" this command does follow it creates a hash string from the first command then it sends the output to the next command i.e. "`cut -d ' ' -f 1`" which cleans the output from the first command and removes the space delimiter and returns the first field i.e. the value for '$mytarget' now if check the file named after this ASCII text we will have the password for our next level bandit23.

```
bandit22@bandit:~$ cd /etc/cron.d
bandit22@bandit:/etc/cron.d$ ls
behemoth4_cleanup  clean_tmp  cronjob_bandit22  cronjob_bandit23  cronjob_bandit24  e2scrub_all
bandit22@bandit:/etc/cron.d$ cat conjob_bandit23
cat: conjob_bandit23: No such file or directory
bandit22@bandit:/etc/cron.d$ cat cronjob_bandit23
@reboot bandit23 /usr/bin/cronjob_bandit23.sh  &> /dev/null
* * * * * bandit23 /usr/bin/cronjob_bandit23.sh  &> /dev/null
bandit22@bandit:/etc/cron.d$ cat /usr/bin/cronjob_bandit23.sh
#!/bin/bash

myname=$(whoami)
mytarget=$(echo I am user $myname | md5sum | cut -d ' ' -f 1)

echo "Copying passwordfile /etc/bandit_pass/$myname to /tmp/$mytarget"

cat /etc/bandit_pass/$myname > /tmp/$mytarget
```

```
bandit22@bandit:/etc/cron.d$ echo I am user bandit23 | md5sum | cut -d ' ' -f 1
8ca319486bfbbc3663ea0fbe81326349
bandit22@bandit:/etc/cron.d$ cat /tmp/8ca319486bfbbc3663ea0fbe81326349
[redacted]
bandit22@bandit:/etc/cron.d$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Prasad

Level 23 – Bandit23

The instruction for this level is same as last 2 level. So, we do the same process to get to the file '/usr/bin/cronjob_bandit24.sh', this file states that it executes and deletes all scripts with the directory '/var/spool/bandit24/foo' as user bandit23. We use this to our advantage and write a shell script in that directory and get the password required for next level.

```
bandit23@bandit:~$ cd /etc/cron.d
bandit23@bandit:/etc/cron.d$ l
behemoth4_cleanup  clean_tmp  cronjob_bandit22  cronjob_bandit23  cronjob_bandit24  e2scrub_all
bandit23@bandit:/etc/cron.d$ cat cronjob_bandit24
@reboot bandit24 /usr/bin/cronjob_bandit24.sh &> /dev/null
* * * * * bandit24 /usr/bin/cronjob_bandit24.sh &> /dev/null
bandit23@bandit:/etc/cron.d$ cat /usr/bin/cronjob_bandit24.sh
#!/bin/bash

shopt -s nullglob

myname=$(whoami)

cd /var/spool/"$myname"/foo || exit
echo "Executing and deleting all scripts in /var/spool/$myname/foo:"
for i in * .*;
do
    if [ "$i" != "." ] && [ "$i" != ".." ];
    then
        echo "Handling $i"
        owner="$(stat --format "%U" "./$i")"
        if [ "${owner}" = "bandit23" ] && [ -f "$i" ]; then
            timeout -s 9 60 "./$i"
        fi
        rm -rf "./$i"
    fi
donebandit23@bandit:/etc/cron.d$
```

But we can't create a shell script there so we need to create a temporary directory which will have the shell script and then copy it to the mentioned directory. The commands to do so are mentioned below.

First, we create a temporary directory and provide full permission so that cron can access and execute the shell file within. Then we change our directory to the temporary directory, and write the shell script using nano editor.

```
#!/bin/bash


cat /etc/bandit_pass/bandit24 > /tmp/pass24/pass24.txt

chmod 777 /tmp/pass24/pass24.txt
```

This script takes the password which is stored in file 'bandit24' and creates a file in the temporary directory named 'pass24.txt' (The names used can be changed according to what we wish) as we currently user bandit24. Then it provides full permission to that file as well. Later we can view what the password for the next level is using the command "`cat pass24.txt`".

Prasad

```
bandit23@bandit:/etc/cron.d$ mkdir /tmp/pass24
bandit23@bandit:/etc/cron.d$ chmod 777 /tmp/pass24
bandit23@bandit:/etc/cron.d$ cd /tmp/pass24
bandit23@bandit:/tmp/pass24$ nano script.sh
Unable to create directory /home/bandit23/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

bandit23@bandit:/tmp/pass24$ nano script.sh
Unable to create directory /home/bandit23/.local/share/nano/: No such file or directory
It is required for saving/loading search history or cursor positions.

bandit23@bandit:/tmp/pass24$ chmod 777 script.sh
bandit23@bandit:/tmp/pass24$ ls
script.sh
bandit23@bandit:/tmp/pass24$ cat script.sh
#!/bin/bash

cat /etc/bandit_pass/bandit24 > /tmp/pass24/pass24.txt
chmod 777 /tmp/pass24/pass24.txt
bandit23@bandit:/tmp/pass24$ cp script.sh /var/spool/bandit24/foo
bandit23@bandit:/tmp/pass24$ ls
pass24.txt  script.sh
bandit23@bandit:/tmp/pass24$ cat pass24.txt
[redacted]
bandit23@bandit:/tmp/pass24$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Prasad N

Level 24 – Bandit24

The instructions for this level are that a daemon is listening on port 30002 and will give you the password for bandit25 if given the password for bandit24 and a secret numeric 4-digit pin code. There is no way to retrieve the pin code except by going through all of the 10000 combinations, called brute-forcing. So, we need to write a shell script which will perform this brute forcing:

```
#! /bin/bash

pass = “password for this level”

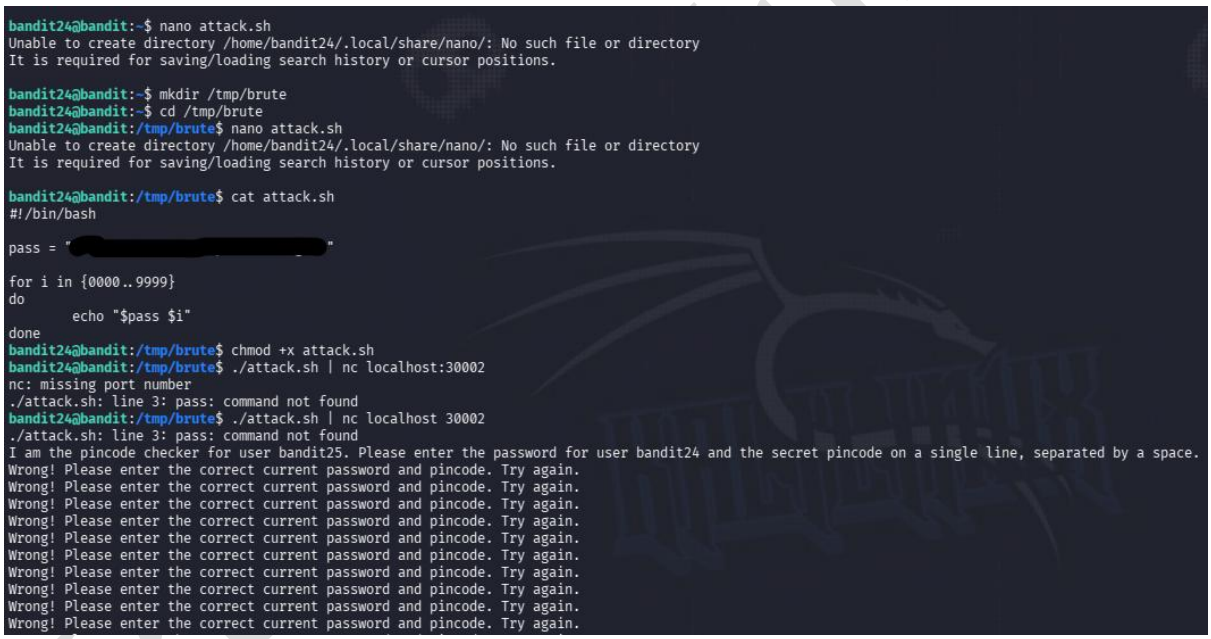
for I in {0000..9999}

do

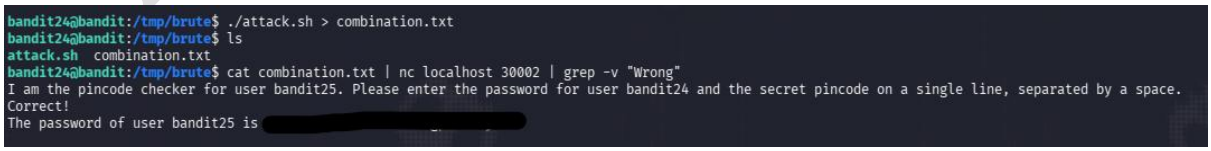
        echo “$pass $i”

done
```

This script will perform brute force without a pin using only the password this level on every port till ports 10000. This script can be executed using command “`./attack.sh | nc localhost 30002`”. This will then show us output for each port then we can search for the correct one and get the password bandit25.



Instead of manual searching we can also do; store the output of the command into a file and use the ‘grep’ command to find the correct output i.e. the password for bandit25.



Level 25 – Bandit25

The instruction for this level is that the shell for user bandit26 is not '/bin/bash', but something else. Find out what it is, how it works and how to break out of it. So, we check for the public that is present in this level and can use it login to bandit26.

```
bandit25@bandit:~$ ls
bandit26.sshkey
bandit25@bandit:~$ cat bandit26.sshkey
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAnaAnMX2t9hzZCs5DGxtAm+ul93mVDbs8d4kmrcIblIqUd/Xl2Ain
Q2Lo8eHrG+z40alkwkUBigyS+WCUgTzLoSiYqN56Xc7fdqWCyWrHkWajejNtm0b3×0bHc/
0lGzq/AtD64DHoHy9HEZ8BLUaF+0ZBguX9pdEXz05B4hFOyC1TuCUYvxgH+jL4ZlxJ8CN0
Dj/1Sef+GBegTPjBXOHzulK4XM35M6tUvmC95Zyx8v6hsxTLSsJLny4RG2+jl9zRgZiHds
O+Pi0EPPqrP/qzSOz6+0noC/dKTTiKOXur3AJdqjZM8bEStdmuG79XRjqCv1YKVV82fMEy
+wP9QkFtMbNvx7XFdJuXY8ST1Au3E40WIeGS2X6cQHhhPZzBtqxSSF4fSQl3VtAso9LVTj
nTZtAFIL9tPImb9U+BKdJsgw3HeSs1MmTzrXI81ejp6VliL1rktywUlPr8a6kH6A14a1JJ
en4ug+KUi3/UYvCPVzXjmdXB0Gd7×1Wnkq4m/4fPAAAFiAqgp+4KoKfuAAAAB3NzaC1yc2
EAAAGBAJ2gJzF9rfYc2QrOQxsbQJvrpfd5lQ27PHeJJq3CG5SKlHf15dgIp0Ni6PHh6xvs
+NGpZMJFAYoMkvlglIE8y6EomKjeel3O33algslqx5Fmo3ozbZtG98dGx3P9JRs6vwLQ+u
Ax6B8vRxGfAS1GhftGQYLl/aXRF89OQeIRTsgtU7glGL8YB/oy+GZcSfAjdA4/9Unn/hgX
oEz4wVzh87pSuFzN+TOrVL5gveWcsfL+obMUy0rCS58uERtvo5fc0YGYh3bDvj4tBDz6qz
/6s0js+vtJ6Av3Sk04ijl7q9wCXao2TPGxErXZrhu/V0Y6gr9WClVfNnzBMvsD/UJBbTGz
b8e1xXSbl2PEk9QLtxONFiHhktl+nEB4YT2cwbasUkheH0kJd1bQLKPS1U4502bQBSC/bT
yJm/VPgSnSbIMNx3krNTJk861yPNXo6elZYi9a5LcsFJT6/GupB+gNeGtSSXp+LoPilIt/
1GLwj1c145nVwdBne8dVp5KuJv+HzwAAAAMBAAEAAAGABjaoQpPAAX+lDFra23z5PnVflz
Jg5To/IIcriN8Xrzvk9mEM0Ufc5u8g7n7jFAGXZoNC7Poe07Leny35LKp02jBW6xXMV1nA
4edMWkDSvkNuFk32ZdkoCik/iUAcOTTZWHTKLu1tR1H+WqCc09XI/t0H1u9o4NotyPSHoz
ADeU0wHfPmH4tyZQ5+dDzN9DMfSumnF/fgC7+nPyqoS4YsrNZ3E78tH6YDDlK3qaF4zbvz
4rK+kxD/9GyyvsvXPVlIskHpJvWnhVa7vzk0EAZ02P4f0FSBImDZ5NUnF1T+h3eLQlj2XM
BgzJAsLg4o/DAj/3yQtbcXhKIeq2dLc0cJwHz1e/t7BFC1GjwxXVOVVsw5aKJJOZmgHKpn
Kc393JQIMnzDS4HTIoa4DZSnX00q7MemRdPQV3rCHlwsIywSW5CrM6eR0xsphhK3WEpkV5
rU3zkidT0z2Efi1Qs3E2fIuH3okmWN8gZ6zWAgBJHJMlfl/ij3s44vdt3VSKcpUMKBAAAA
wQCrI/HTiT0XanBM9cAwvnv9wUBeCeLbUJa/r7LOQu8QrrNu9Y/6KUy9gfEEP1saHOiRNN
LNtxPpcKxYOGqZwrSzAP793raGg7Q3/OIghLPgJaRpfGlS4swxrvqG/ddG4E6+mY8BoRWy
VjFW5pKxwGCPwpkLvvPOA/0ukC1O1Spc6v1wwSvKcPKBhEQZ6fUtpWLdTn7mKaTHXhVLHv
WOTyJ8cg3fI36/UkagsiCFUoRamzY6OwZTSeS/X2SGkEvjsBYAAADBAMl3Rzmw4c/l5AjC
bmKlW/EVDTzeLe4UGePd4PMoJu1YoC8u3sKzv/rbz8vt6tHlwUsvI5NubwJ1pUaIEcL/br
FbAvkuV9NYUQqMW0qsgecXmQV2MBhZbKwLl8hq9shav6YL4RAjFIl3Xsx9ngH6BHXx51zI
2LUZXq1EZtw6wdc9glt14/w4m+N+5WRaT641O19o8yKhAHYqnwSTNUOFL8X4gkUCTQbTne
Ochzn4Q56gP2boTN2frB8hbY3j1BLMHwAAAMEAyErttPP7OJnNrYdbQ3YKVOM69o+YXa0y
```

Then we can check the passwd file which shows the shell for each level or user and here it shoes that the shell for bandit26 is '/bin/showtext'.

```
bandit25@bandit:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
```

```
bandit22:x:11022:11022:bandit level 22:/home/bandit22:/bin/bash
bandit23:x:11023:11023:bandit level 23:/home/bandit23:/bin/bash
bandit24:x:11024:11024:bandit level 24:/home/bandit24:/bin/bash
bandit25:x:11025:11025:bandit level 25:/home/bandit25:/bin/bash
bandit26:x:11026:11026:bandit level 26:/home/bandit26:/usr/bin/showtext
bandit27:x:11027:11027:bandit level 27:/home/bandit27:/bin/bash
bandit28:x:11028:11028:bandit level 28:/home/bandit28:/bin/bash
bandit29:x:11029:11029:bandit level 29:/home/bandit29:/bin/bash
bandit3:x:11003:11003:bandit level 3:/home/bandit3:/bin/bash
bandit30:x:11030:11030:bandit level 30:/home/bandit30:/bin/bash
bandit31:x:11031:11031:bandit level 31:/home/bandit31:/bin/bash
bandit32:x:11032:11032:bandit level 32:/home/bandit32:/home/bandit32/uppershell
bandit33:x:11033:11033:bandit level 33:/home/bandit33:/bin/bash
```

To access this shell, we can use the public key and log in but it states that the connection is terminated. But there is a roundabout about this that if we minimize the terminal to small enough where the text can't fit the screen. Then the pager stops and we can enter the 'vim' editor by pressing 'v'. Then we type "`:set shell=/bin/bash`" and hit enter, after this we type "`:shell`" to enter the command prompt and check the file "/etc/bandit_pass/bandit26" which stores password for bandit26.

```
~
~
~
~
~
~
:set shell=/bin/bash
```

```
~
:shell
bandit26@bandit:~$ cat /etc/bandit_pass/bandit26
[redacted]
bandit26@bandit:~$
```

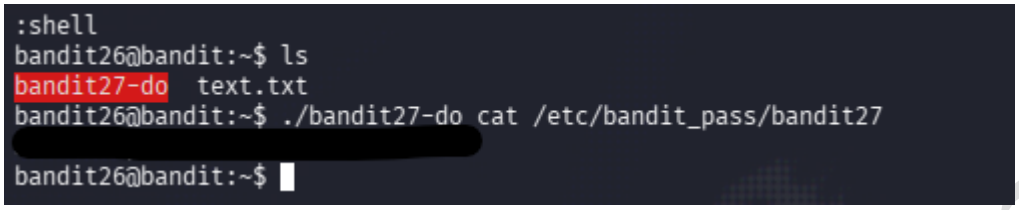
After this exit using command ":qa!" which will exit the editor and hit enter a few times to get to normal.

```
~
~
~
:qa!
```

Prasad Nikam

Level 26 – Bandit26

As we did previously enter the vim editor using the method and check the directory as the instruction the file is there itself. The directory consists of 2 files first is a do file which has the owner rights of bandit27, we can use this to access the file from "/etc/bandit_pass/bandit27" using command "`./bandit27-do cat /etc/bandit_pass/bandit27`", which will show us the password for next level i.e. bandit27.



Prasad Nikam

Level 27 – Bandit27

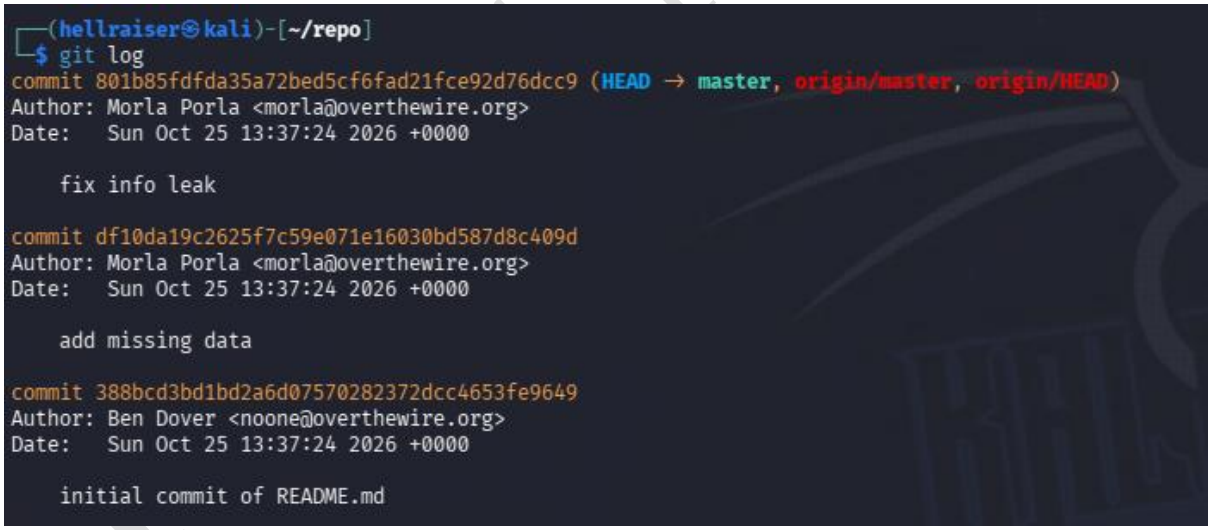
The instruction for this level is that the password for the next level is in a GitHub repository "ssh://bandit27-git@bandit.labs.overthewire.org/home/bandit27-git/repo" upon connected using port 2220. So, we can copy the repository using command "`git clone ssh://bandit27-git@bandit.labs.overthewire.org:2220/home/bandit27-git/repo`" and check for the password within the file.



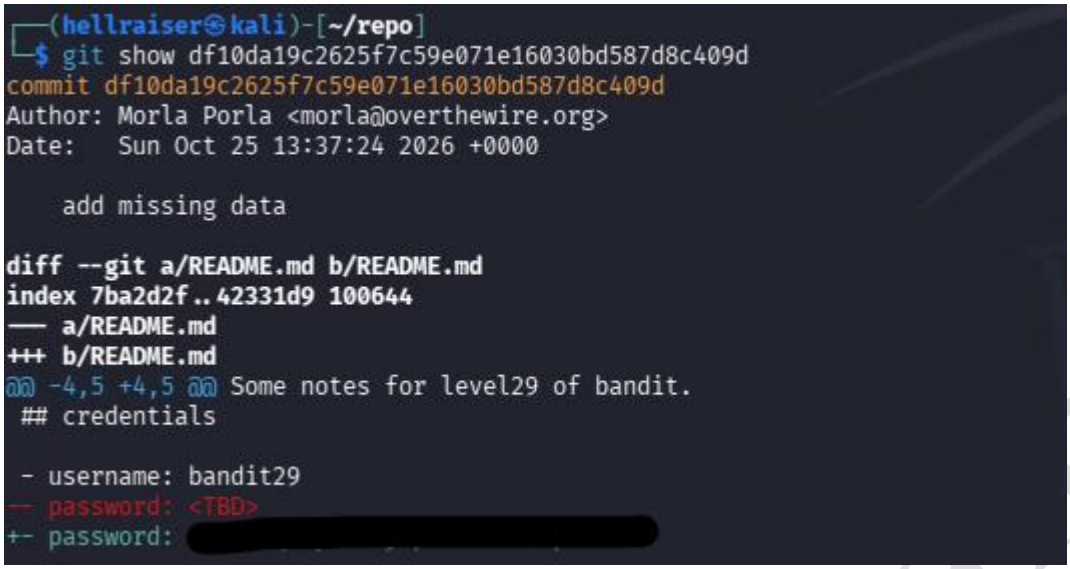
Level 28 – Bandit28

The instruction for this level is same as previous. So, we will copy the git repository into the file "Repo" using "`git clone ssh://bandit28-git@bandit.labs.overthewire.org:2220/home/bandit28-git/repo`" then we will check the file present in directory 'Repo'. The file contains password but it is hidden. So, we can check the log for the Git using "`git log`".





The log shows three activities; first one would be the creation of 'README.md' file then the second one would be the writing of username and password. So, if we check that report we might have the password. Whereas the third one would be hiding the password. We can see the commit using command "git show df10da19c2625f7c59e071e16030bd587d8c409d" which will show the password for the level bandit29.



Prasad Nikam

Level 29 – Bandit29

This level is same as previous 2 levels. We log into git using "`git clone ssh://bandit29-git@bandit.labs.overthewire.org:2220/home/bandit29-git/repo`" and copy the repository in directory 'repo'. The file in this level shows that the password for the next level is not in production. So, we for check the branches which might have the password for the next level as the password is not in head using command "`git branch -a`". We will check the first branch that is 'dev' using command "git checkout dev" and check the files within. Upon checking the file inside this branch, it does contain the password we are looking for i.e. for bandit30.

```
┌──(hellraiser㉿kali)-[~/repo]
└─$ ls
README.md

┌──(hellraiser㉿kali)-[~/repo]
└─$ cat README.md
# Bandit Notes
Some notes for bandit30 of bandit.

## credentials

- username: bandit30
- password: <no passwords in production!>


┌──(hellraiser㉿kali)-[~/repo]
└─$ git branch -a
* master
  remotes/origin/HEAD → origin/master
  remotes/origin/dev
  remotes/origin/master
  remotes/origin/sploits-dev

┌──(hellraiser㉿kali)-[~/repo]
└─$ git checkout dev
branch 'dev' set up to track 'origin/dev'.
Switched to a new branch 'dev'
```

```
┌──(hellraiser㉿kali)-[~/repo]
└─$ ls
code  README.md

┌──(hellraiser㉿kali)-[~/repo]
└─$ cat README.md
# Bandit Notes
Some notes for bandit30 of bandit.

## credentials

- username: bandit30
- password: [redacted]
```

Level 30 – Bandit30

This level is same as well; we log onto git and copy the repository using command "`git clone ssh://bandit30-git@bandit.labs.overthewire.org:2220/home/bandit30-git/repo`". Then we check the readme.md file but it contains nothing. But following the logic according to the previous 3 levels the next one would be 'tag'. So, we use the command "`git tag`" which show us available tags. The command shows the tag 'secret' upon reading the secret tag. It has the password for the next level i.e. bandit31.



Prasad Nikam

Level 31 – Bandit31

The instruction for this level is that we have to push the file ‘key.txt’ to git to access the password. So, we need to create a file named ‘key.txt’ and print “May I come in?” in it. Then we need to add it to git. Also to push the file to git we need a username and email which is stored in the machine which could be logged in using the previous ssh command i.e. “`ssh -p 2220 bandit31 bandit.labs.overthewire.org`”. The contents are stored in “.gitconfig” and we need to assign them in the repo and commit the changes to git. The we need push the file using “`git push`” which will return the password.



```
┌──(hellraiser㉿kali)-[~]
└─$ cd repo/

┌──(hellraiser㉿kali)-[~/repo]
└─$ ls
README.md

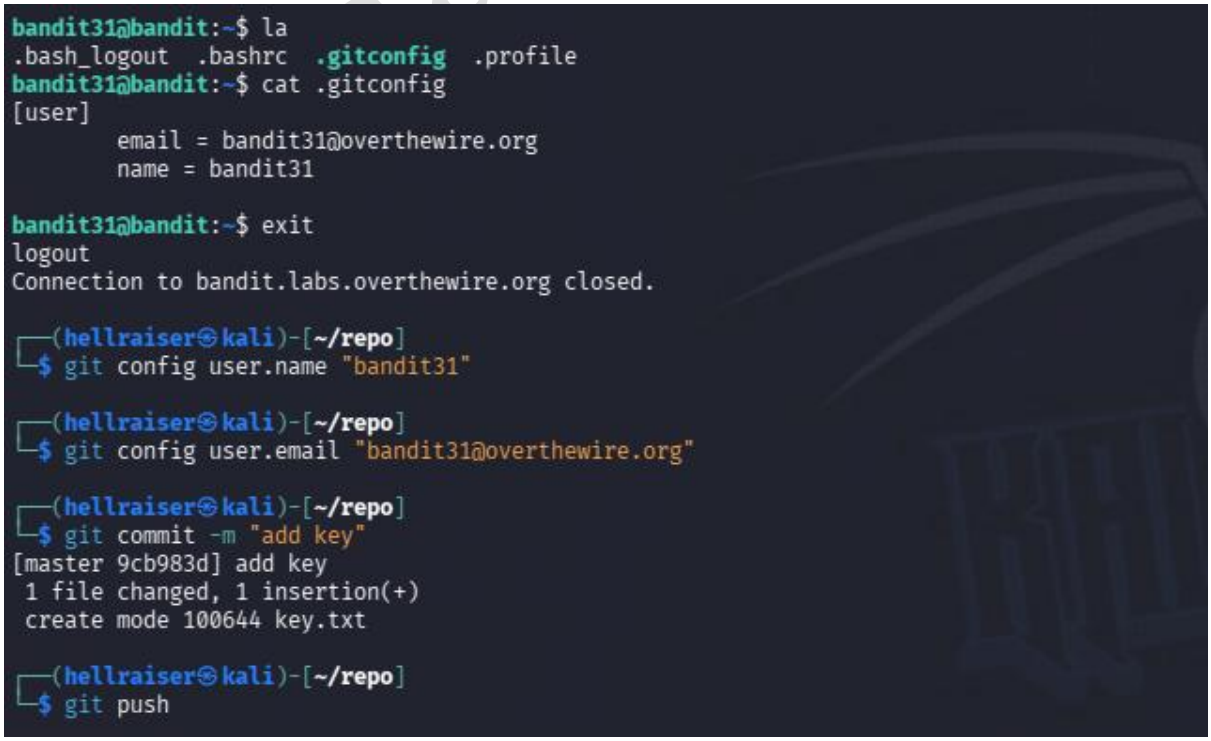
┌──(hellraiser㉿kali)-[~/repo]
└─$ cat README.md
This time your task is to push a file to the remote repository.

Details:
    File name: key.txt
    Content: 'May I come in?'
    Branch: master


┌──(hellraiser㉿kali)-[~/repo]
└─$ echo "May I come in?" > key.txt

┌──(hellraiser㉿kali)-[~/repo]
└─$ git add -f key.txt

┌──(hellraiser㉿kali)-[~/repo]
└─$ ssh -p 2220 -l bandit31 bandit.labs.overthewire.org
                         _                     _ _ _
                        | |__   __ _ _ __   __| (_) |_
                        | '_ \ / _` | '_ \ / _` | | __|
                        | |_) | (_| | | | | (_| | | |_
                        |_.__/ \__,_|_| |_|\__,_|_|\__|
```



```
bandit31@bandit:~$ la
.bash_logout  .bashrc  .gitconfig  .profile
bandit31@bandit:~$ cat .gitconfig
[user]
        email = bandit31@overthewire.org
        name = bandit31

bandit31@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.

┌──(hellraiser㉿kali)-[~/repo]
└─$ git config user.name "bandit31"

┌──(hellraiser㉿kali)-[~/repo]
└─$ git config user.email "bandit31@overthewire.org"

┌──(hellraiser㉿kali)-[~/repo]
└─$ git commit -m "add key"
[master 9cb983d] add key
 1 file changed, 1 insertion(+)
 create mode 100644 key.txt

┌──(hellraiser㉿kali)-[~/repo]
└─$ git push
```

Level 32 – Bandit32

According to the instruction this is a special level, where we are inside a uppercase shell (which executes everything in uppercase automatically even if it is in lowercase), so we can not run normal commands in this shell instead we can enter “$0” which will launch a normal shell inside this uppercase shell and we simply check the password file as the user bandit32 is the group for this file which can read the file.

```
WELCOME TO THE UPPERCASE SHELL
>> $0
$ cat /etc/bandit_pass/bandit33
[redacted]
$ exit
>> exit
sh: 1: EXIT: Permission denied
>> ^CConnection to bandit.labs.overthewire.org closed.
```

Prasad Nikam

Level 33 – Bandit33

This is the last level which has a message for us which Congratulates us for completing the Bandit Wargame of overthewire.org.

```
bandit33@bandit:~$ ls
README.txt
bandit33@bandit:~$ cat README.txt
Congratulations on solving the last level of this game!

At this moment, there are no more levels to play in this game. However, we are constantly working
on new levels and will most likely expand this game with more levels soon.
Keep an eye out for an announcement on our usual communication channels!
In the meantime, you could play some of our other wargames.

If you have an idea for an awesome new level, please let us know!
bandit33@bandit:~$ exit
logout
Connection to bandit.labs.overthewire.org closed.
```

Prasad Nikam